

jxnl.co

@jxnlco



Systematically Improving RAG Applications

Week 6: Product**

Jason Liu and Dan Becker

Overview

Three goals for today

1. Makes sure we're taking actions to collect feedback
2. Expand on what is possible with streaming
3. Give you a small set of prompting and ux tips to improve satisfaction and quality

Consider this mostly a survey of other techniques that I apply. This does not fit neatly into the other sections this course.

Agenda

Feedback

Streaming

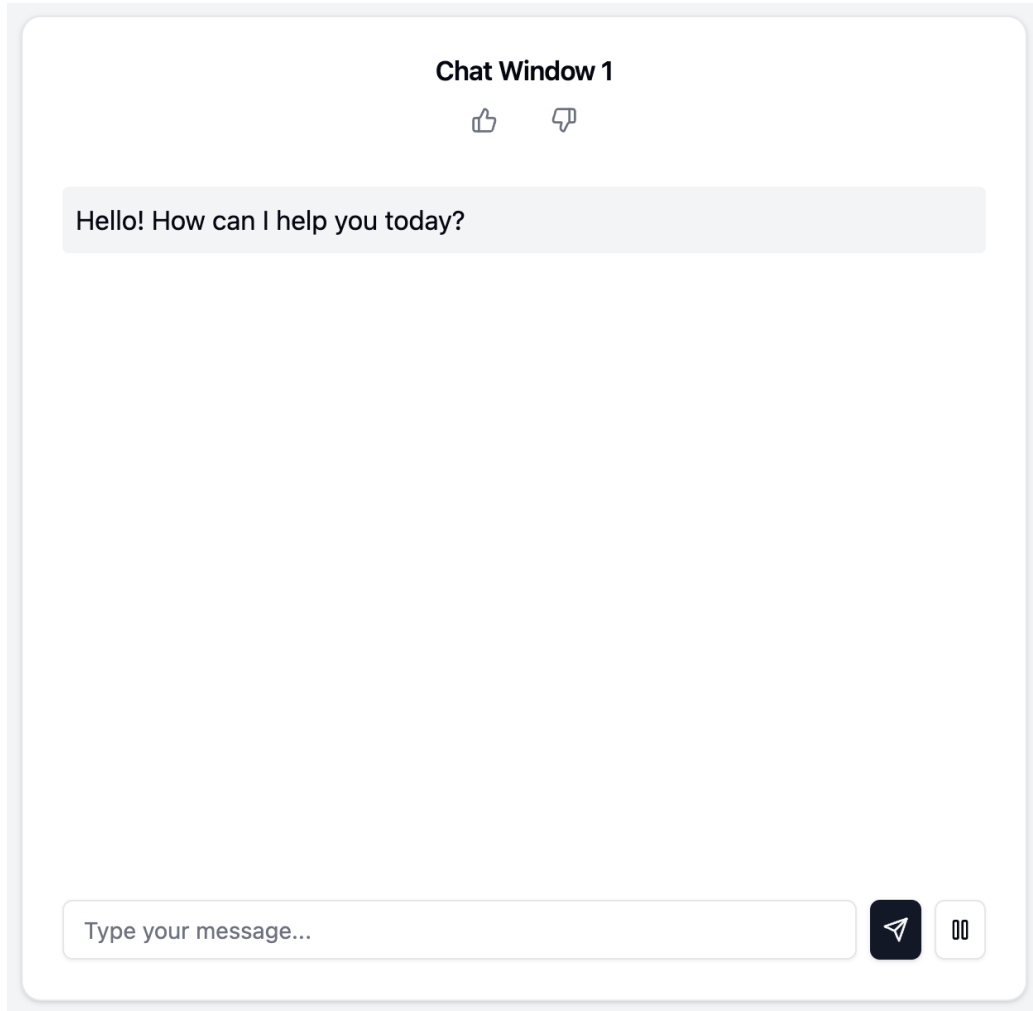
Prompting

Course conclusion

- Q&A

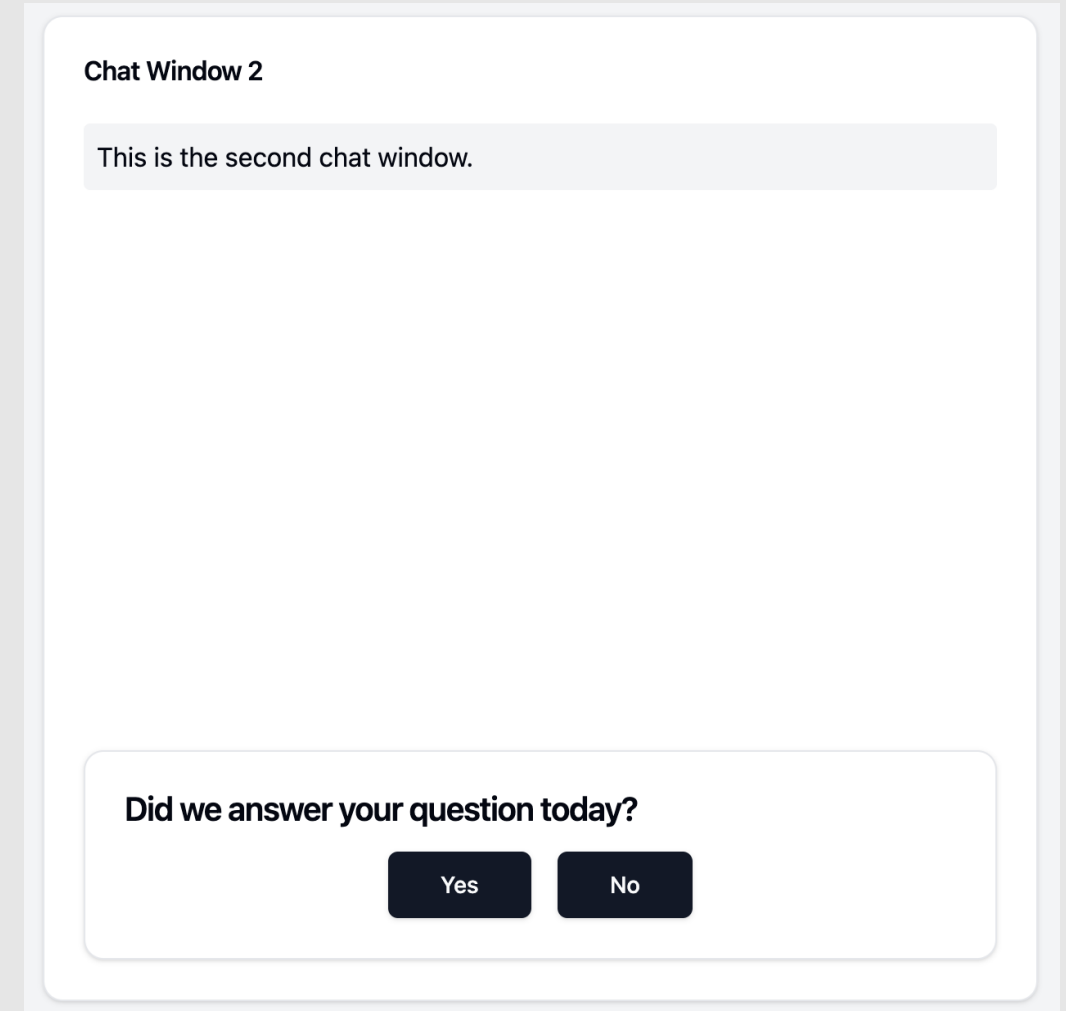
Getting user feedback is the second most important thing you can be doing after looking at your input data

Don't be subtle



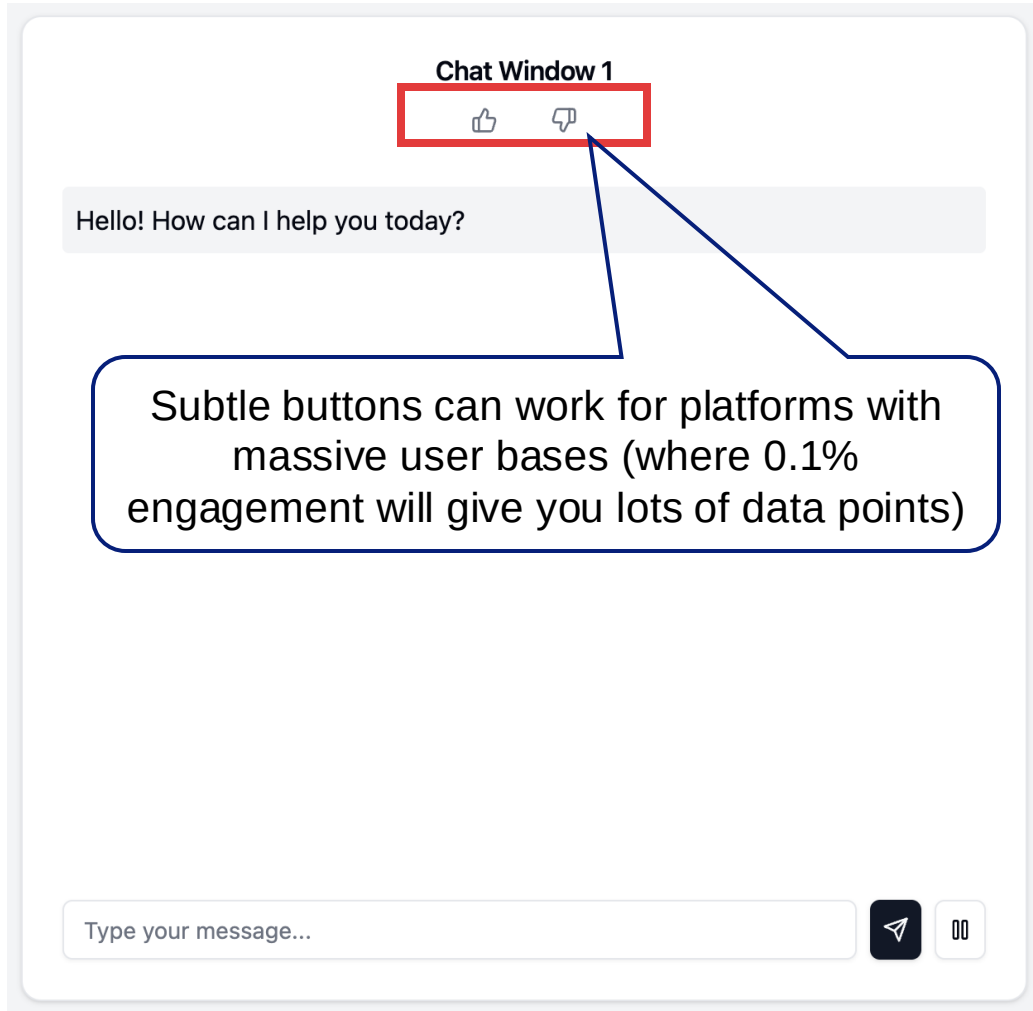
@jxnico

<https://claude.site/artifacts/d57936fe-03c1-4815-8511-cbdb507d6d9c>



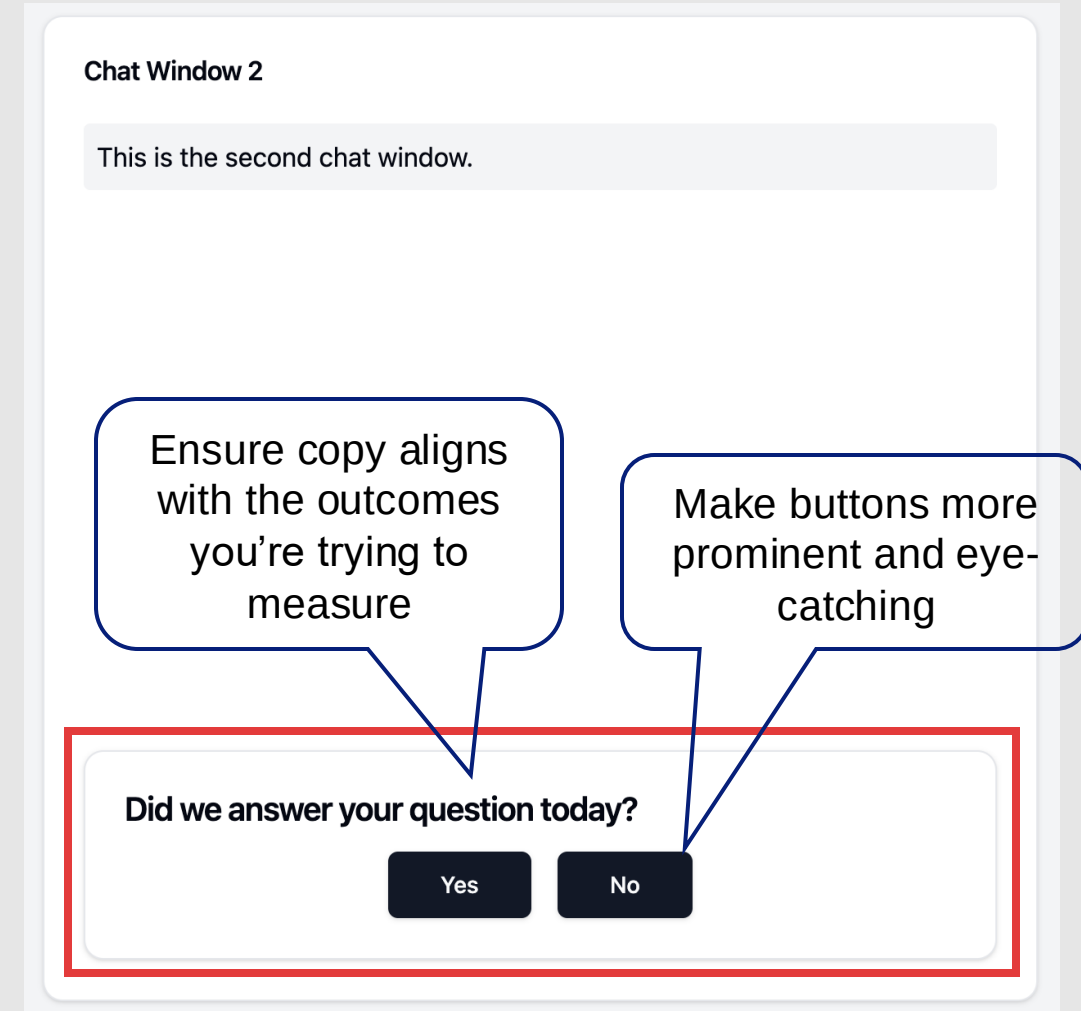
maven.com/applied-llms/rag-playbook

Don't be subtle



@jxnico

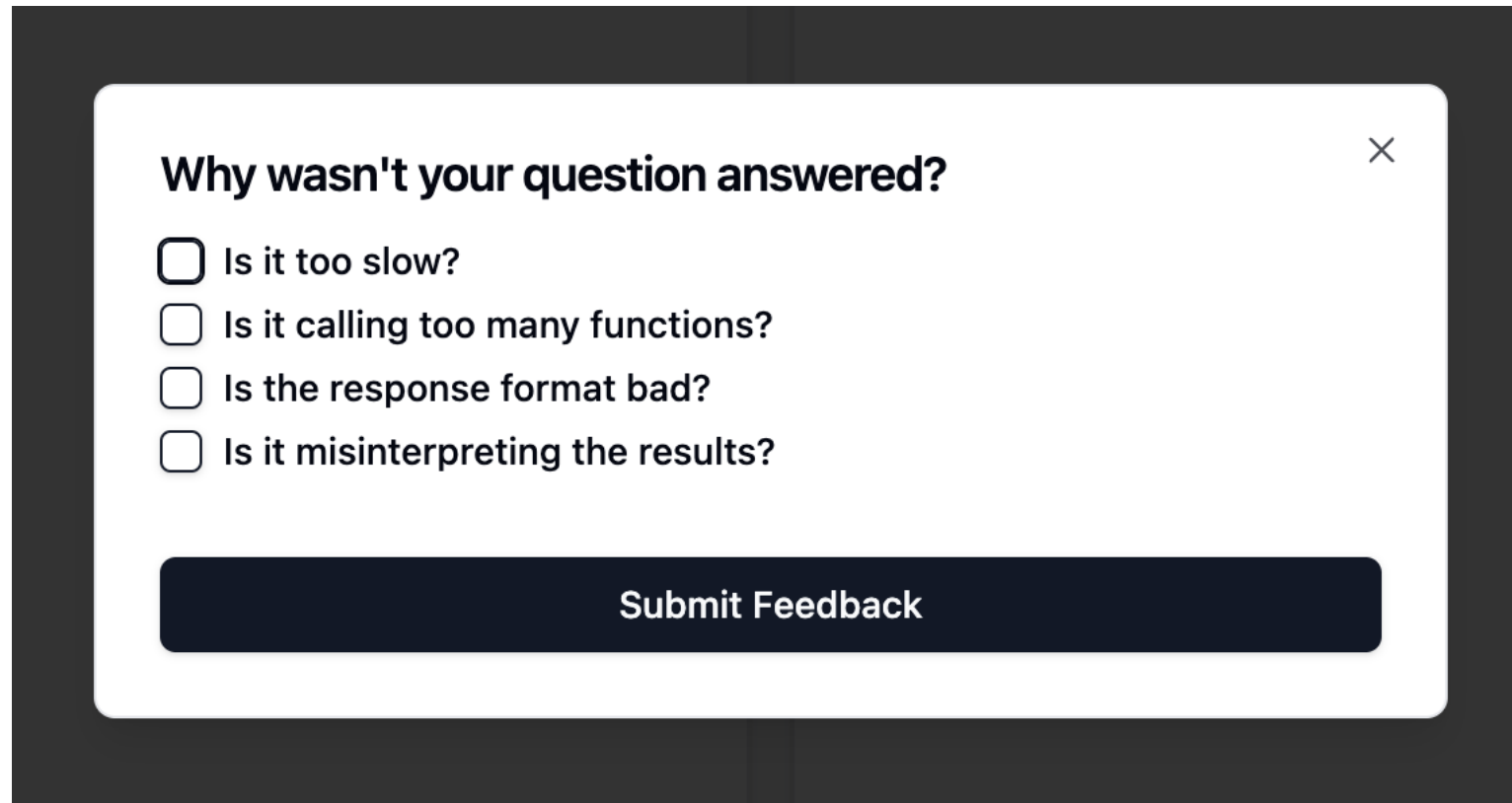
<https://claude.site/artifacts/d57936fe-03c1-4815-8511-cbdb507d6d9c>



maven.com/applied-llms/rag-playbook

Don't be subtle

<https://claude.site/artifacts/d57936fe-03c1-4815-8511-cbdb507d6d9c>



Why wasn't your question answered? ×

- ☐ Is it too slow?
- ☐ Is it calling too many functions?
- ☐ Is the response format bad?
- ☐ Is it misinterpreting the results?

Submit Feedback

All this feedback can be used as part of your segmentation and exploration.

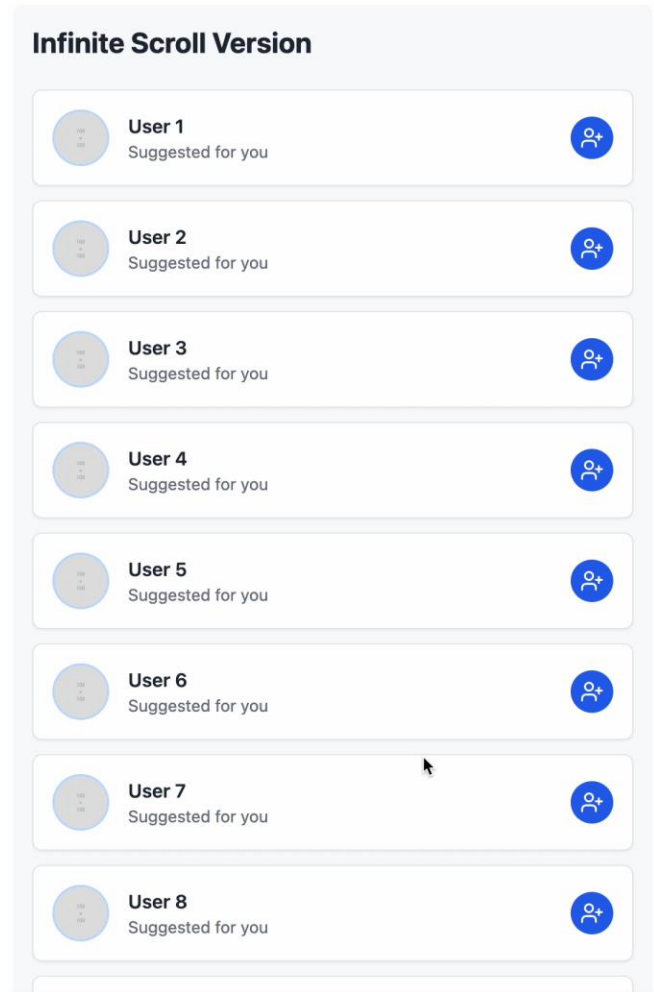
Question:

- Are there certain question segments that fall victim to these failure modes?
- Could we predict if a question might lead to a failure mode?

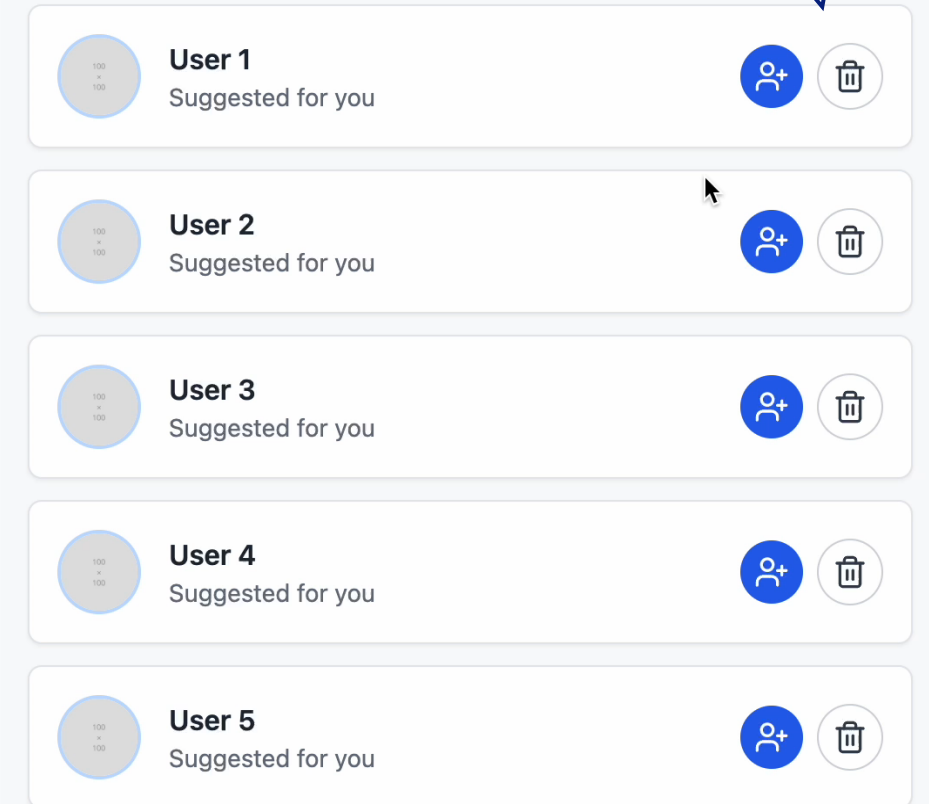
This kind of feedback data gets us dataset for query analysis, but we can also use feedback it to improve our re-rankers and our embedding models.

Finding Hard Negatives

Consider Facebook's people you may know feature



Limited Profiles Version



Requires deleting a profile to see a new one, naturally increasing the amount of data collected

There will be numerous ways to collect positive and negative examples.

I firmly believe that the success of Tinder and Hinge is rooted in the volume of data they collect and their ability to obtain negatives.

- Lightweight Interaction (Swipe)
- Positive and Negative interactions (Like, Dislike)
- Simple objective (Match)

I believe I can do the same thing with citations, allowing us to collect data, while also building more trust in our system

Building Citations To Improve Trust

Based on your schedule, you have a meeting with the marketing team at 2 PM today. Also, you received an important email from Jason about the quarterly report. Would you like me to prepare a summary of the report for your meeting?

✉ Citation 1

📅 Citation 2

Building Citations :

- Allow Preview of Citation text
- ***Allow Delete/Regenerate in order to give negative feedback***

<https://claude.site/artifacts/14cbd428-0382-44b8-9353-1c06d830b8dc>

Building Citations To Improve Trust

Provide a
prompt

```
<task>
  Your task is to provide a summary of the user's schedule and important
  messages, with citations to the original sources. Use the following
  format for citations: (cited text)[citation number].
</task>

<given_information>
  [1] The user has a calendar event: "Marketing Team Meeting" at 2 PM
  today.</item>
  [2] The user received an email from jason@company.com with the subject
  "Quarterly Report Due".</item>
</given_information>

Generate a brief summary that mentions both the meeting and the email,
using citations
Your response should be in JSON format with a "body" field for the main
text and a "citations" array for the citation details.</step>
```

Negative examples: generate citations to build satisfaction and trust (cont'd)

```
<task>
  Your task is to provide a summary of the user's schedule and important
  messages, with citations to the original sources. Use the following
  format for citations: (cited text)[citation number].
</task>

<given_information>
  [1] The user has a calendar event: "Marketing Team Meeting" at 2 PM
  today.</item>
  [2] The user received an email from jason@company.com with the subject
  "Quarterly Report Due".</item>
</given_information>

Generate a brief summary that mentions both the meeting and the email,
using citations
Your response should be in JSON format with a "body" field for the main
text and a "citations" array for the citation details.</step>
```

Provide a
prompt

Return a structured
object

```
1 {
2   "body": "Based on your schedule, you have a (meeting with the marketing team)[2] at 2 PM today.
3   Also, you received an (important email from Jason)[1] about the quarterly report. Would you like
4   me to prepare a summary of the report for your meeting?",
5   "citations": [
6     {
7       "chunk_id": 1,
8       "title": "Email from jason@company.com: Subject: Quarterly Report Due"
9     },
10    {
11      "chunk_id": 2,
12      "title": "Calendar event: Marketing Team Meeting, 2 PM"
13    }
14  ]
15 }
```

Negative examples: generate citations to build satisfaction and trust (cont'd)

```
<task>
  Your task is to provide a summary of the user's schedule and important
  messages, with citations to the original sources. Use the following
  format for citations: (cited text)[citation number].
</task>

<given_information>
  [1] The user has a calendar event: "Marketing Team Meeting" at 2 PM
  today.</item>
  [2] The user received an email from jason@company.com with the subject
  "Quarterly Report Due".</item>
</given_information>

Generate a brief summary that mentions both the meeting and the email,
using citations
Your response should be in JSON format with a "body" field for the main
text and a "citations" array for the citation details.</step>
```

```
{
  "query": ...
  "chunk_ids": ["chunk1", "chunk2", "chunk3"],
  "neg_chunk_ids": ["chunk2"]
}
```

Provide a
prompt

Return a structured
object

```
1 {
2   "body": "Based on your schedule, you have a (meeting with the marketing team)[2] at 2 PM today.
3   Also, you received an (important email from Jason)[1] about the quarterly report. Would you like
4   me to prepare a summary of the report for your meeting?",
5   "citations": [
6     {
7       "chunk_id": 1,
8       "title": "Email from jason@company.com: Subject: Quarterly Report Due"
9     },
10    {
11      "chunk_id": 2,
12      "title": "Calendar event: Marketing Team Meeting, 2 PM"
13    }
14  ]
15 }
```

Use consistent formatting for links and employ
IDs as pointers

Agenda

Collecting feedback

Streaming for better user satisfaction

Prompting

Course conclusion

- Q&A

Streaming overview

Why we should stream:

Lower perceived wait time

- Users perceive animated progress bars as 11% faster, even with equal wait times (Harrison et al.).

Reduce user churn / disengagement

- Users will tolerate up to 8 seconds of waiting if given visual feedback reducing abandonment (Nah).

Increase satisfaction and trust

- Applications with engaging loading screens often report higher user satisfaction scores.
- Facebook discovered that skeleton screens reduced perceived load times, resulting in better user retention and engagement.

Harrison, C., Yeo, Z., & Hudson, S. E. (2010). Faster progress bars: manipulating perceived duration with visual augmentations. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*.

Nah, F. F. (2004). A study on tolerable waiting time: how long are Web users willing to wait?. *Behaviour & Information Technology*.

[@jxnico](#)

We can stream:

- Responses (incl. citations or follow-up questions)
- Arguments of function calls
- Interstitials

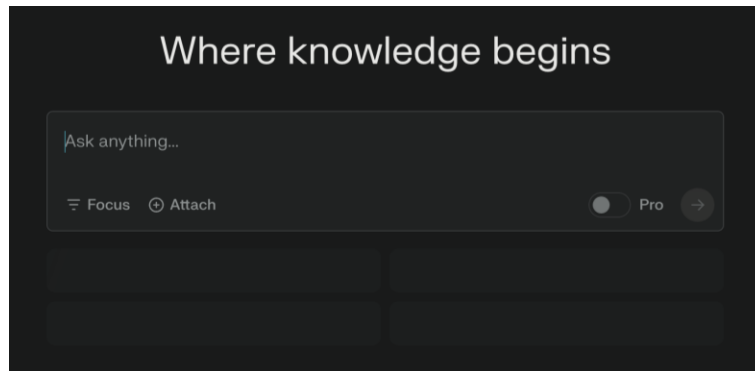
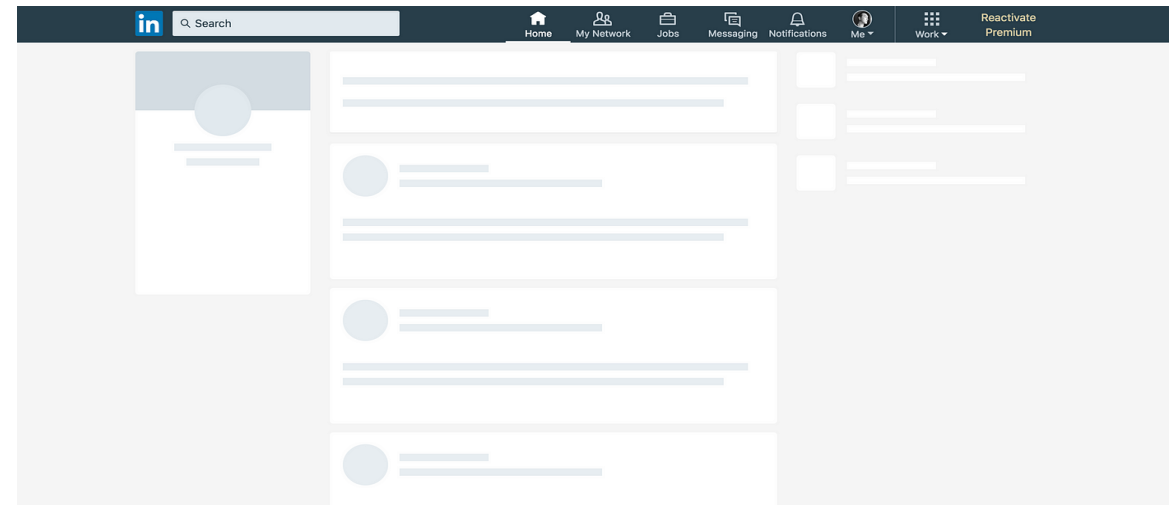
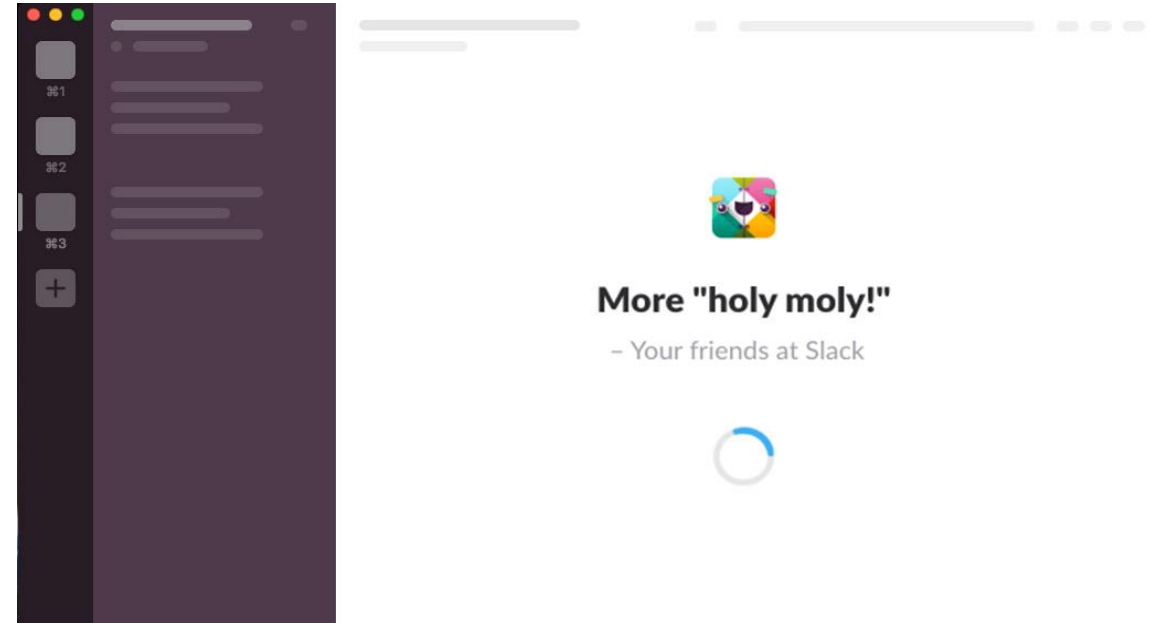
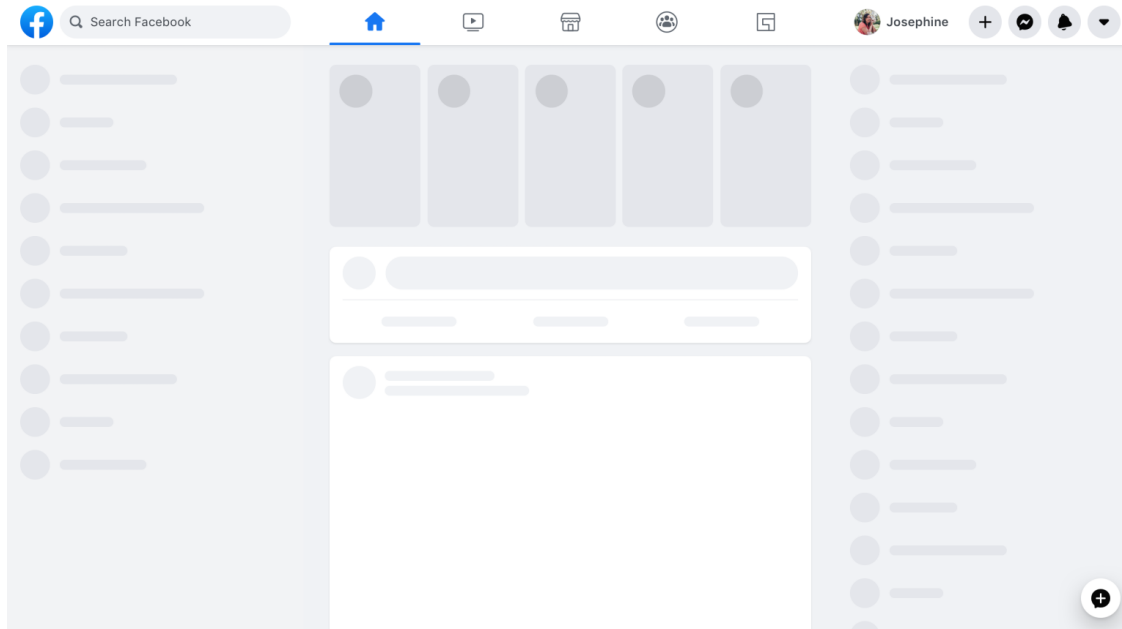
We should assess:

- Track user feedback to understand user demand
- Migrating to streaming is complex and challenging
- Understand how much latency matters in your application

maven.com/applied-llms/rag-playbook

Skeleton screens are everywhere

Once you see them, you cannot unsee them



If you're thinking about it...

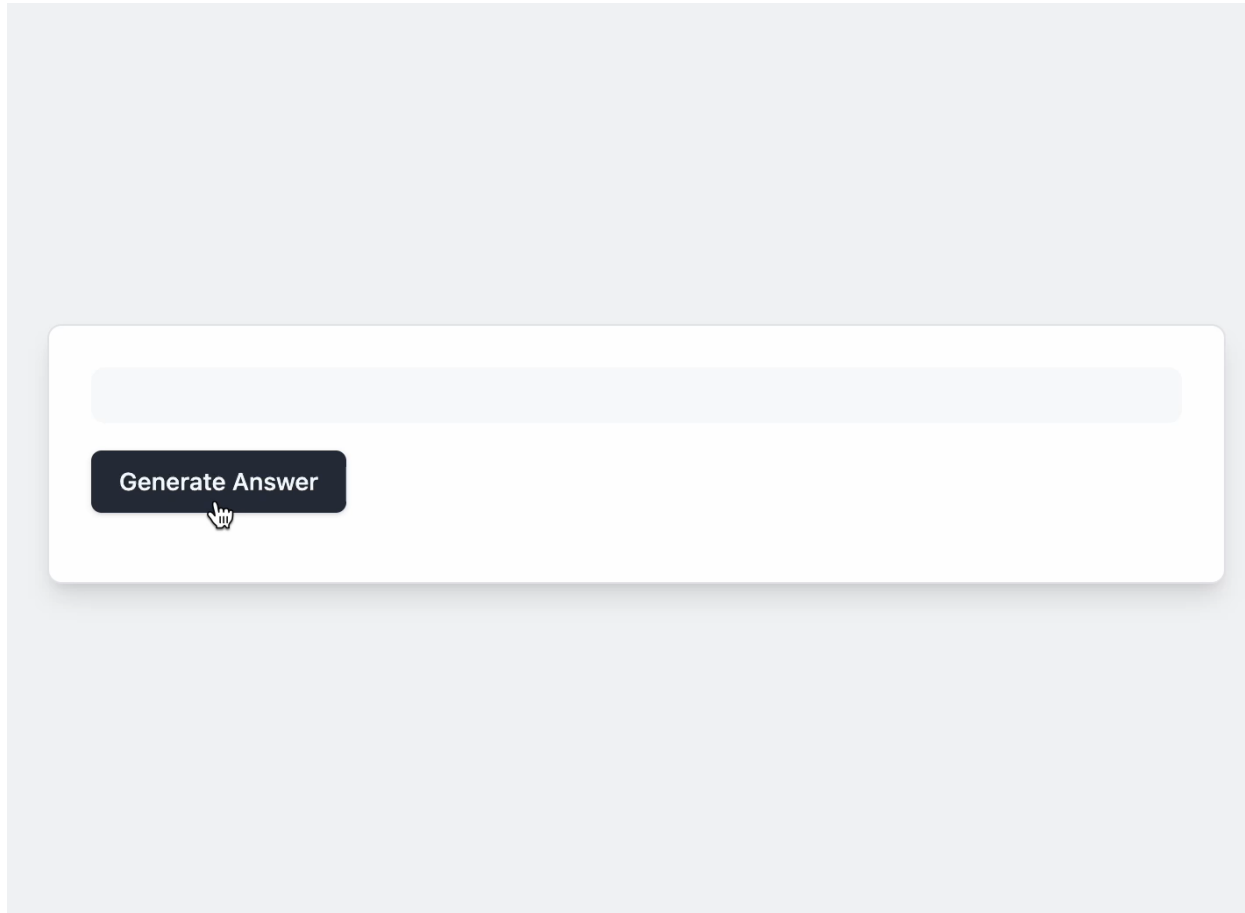
Migrating from non-streaming to streaming is a pain the ass, you either have to build it from the start or it'll take weeks out of your dev cycle to 'upgrade'

It's worth it.

Streaming Overview

1. Stream results and UI
2. Stream tool calls!
3. Stream interstitials to explain latency

Stream and parse the response and other attributes (follow-up actions) in real time



<https://claude.site/artifacts/8644e9fc-939a-4520-8fa2-58f589f929d3>

@jxnico

```
class Citation(BaseModel):
    id: str
    title: str

class RespondToUser(BaseModel):
    content: str
    follow_actions: List[str]
    sources: List[Citation]
```

```
client = instructor.from_openai(client)

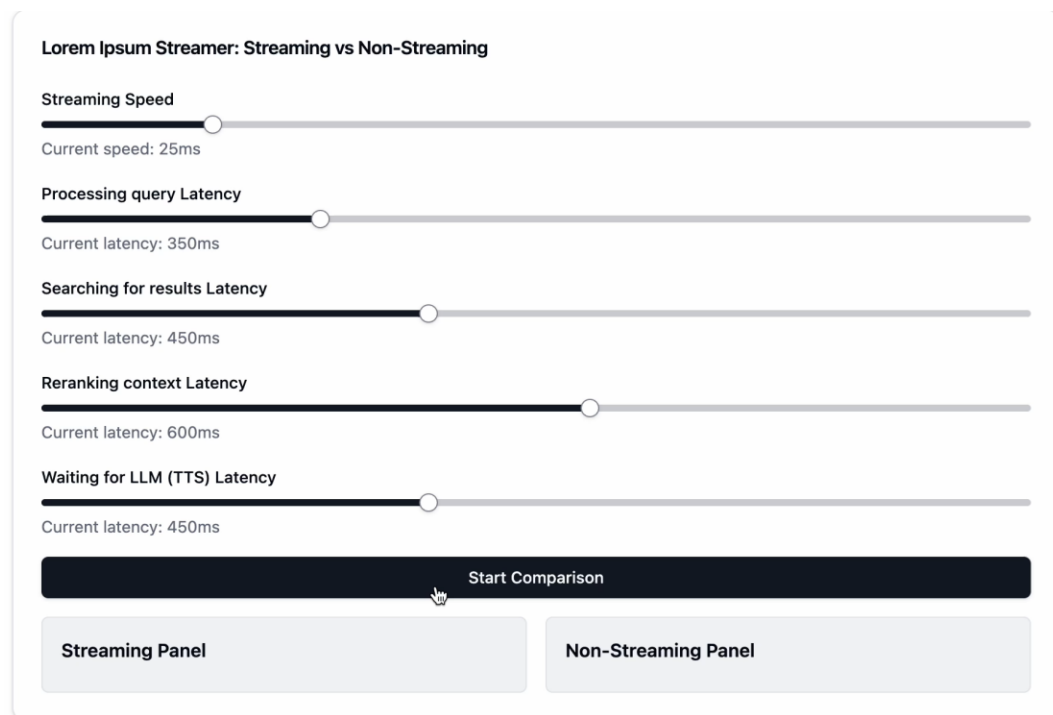
partial_response = client.create_partial(
    messages=[...]
    model=...
    response_model=RespondToUser
)

for response in partial_response:
    yield response
```

maven.com/applied-llms/rag-playbook

If you build a lot of follow-up action type UI, you can build datasets and again use retrieval to few-shot your prompts to take follow-up actions and do the kind of query analysis we've talked about throughout the course.

Interstitials for explainability



@jxnico

<https://claude.site/artifacts/bba5efe9-1d41-49ff-a43f-05d3d349e193>

Use webhooks, server-sent events, or a generator to stream

```
from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse
import random
import asyncio
import json

app = FastAPI()

...

async def sse_stream(request: Request, query: str):
    async def event_generator():
        try:
            # Understand query
            yield json.dumps({"event": "thinking", "data": "..."})
            search = await understand_query(query)
            yield json.dumps({"event": "tool", "data": query_understanding.model_dump()})

            # Search
            yield json.dumps({"event": "searching", "data": "..."})
            search_results = await search.execute()
            yield json.dumps({"event": "search_results", "data": search_results})

            # Generate answer
            answer_stream = await generate_answer(query, search_results)
            for chunk in answer_stream:
                yield json.dumps({"event": "generating", "data": chunk.model_dump()})

            yield json.dumps({"event": "complete", "data": "[DONE]"})
        except Exception as e:
            yield json.dumps({"event": "error", "data": str(e)})

    return EventSourceResponse(event_generator())

@app.get("/query")
async def process_query(request: Request, query: str):
    return StreamingResponse(sse_stream(request, query), media_type="text/event-stream")
```

maven.com/applied-llms/rag-playbook

Stream tool arguments and render in UI

JSON Stream Loading JSON...

Start Streaming

Query Details

Searching for: Edit

Date range: to Edit

Sources: Edit

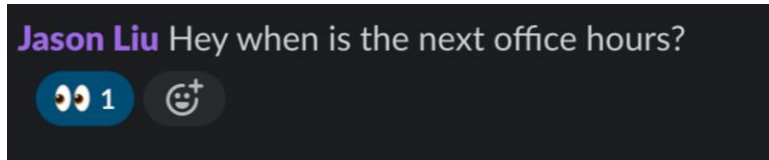
Benefits

- Create UI that allows users to edit and rerun tools
 - This becomes data collection that we can leverage to create better few-shot examples or fine-tuning data.
- Enables collection of better feedback data
- Supports analytics on what questions have additional attributes

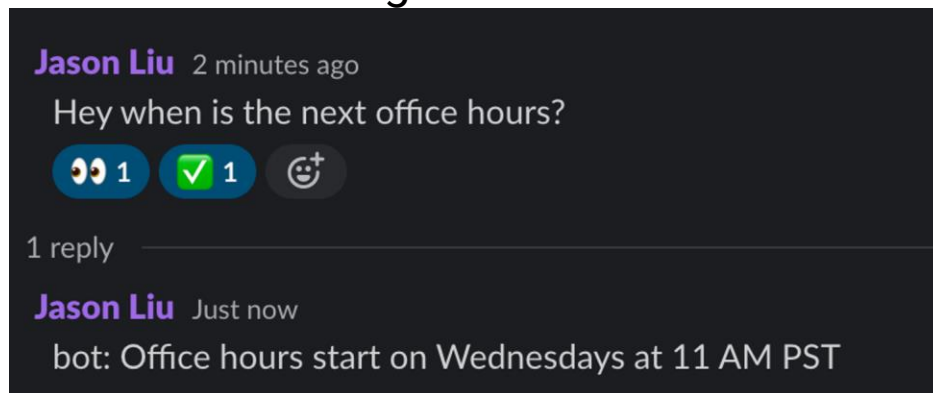
<https://claude.site/artifacts/1a2a1e9a-e5d7-4511-a299-211d70fdeb3b>

Simple Example: Slack Bots

As an interstitial, the **slack integration can react with the eyes emoji** to communicate it has seen the user's message

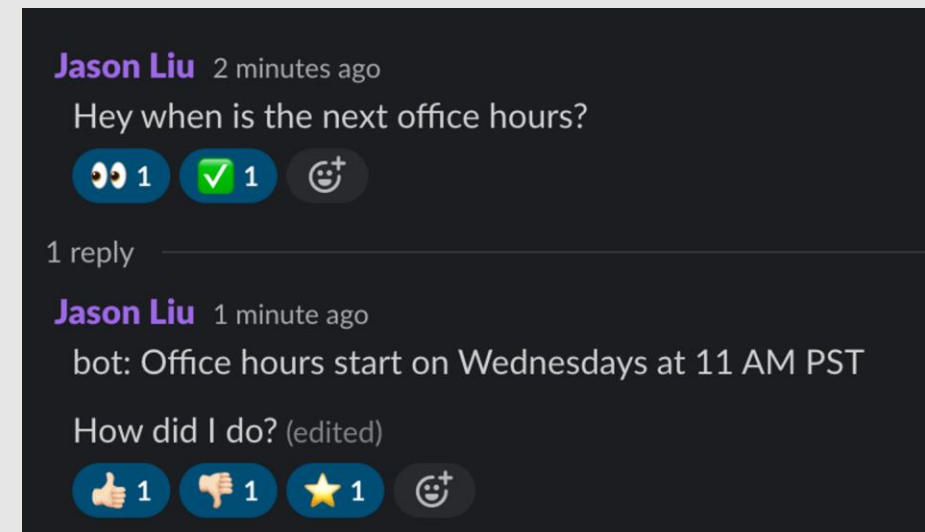


Use a checkmark to communicate the bot has finished answering



@jxnico

Pre-fill emoji reactions (thumbs-up, thumbs-down, star) to communicate to the user there are alternative ways to provide feedback



This would be saved as an approved question-answer pair which could be used in few-shot examples

maven.com/applied-llms/rag-playbook

If you see great examples of using interstitials and streaming I would love to see them on Slack in #random or even in the Zoom chat.

Agenda

Collecting feedback

Streaming

Prompting

Course conclusion

Q&A

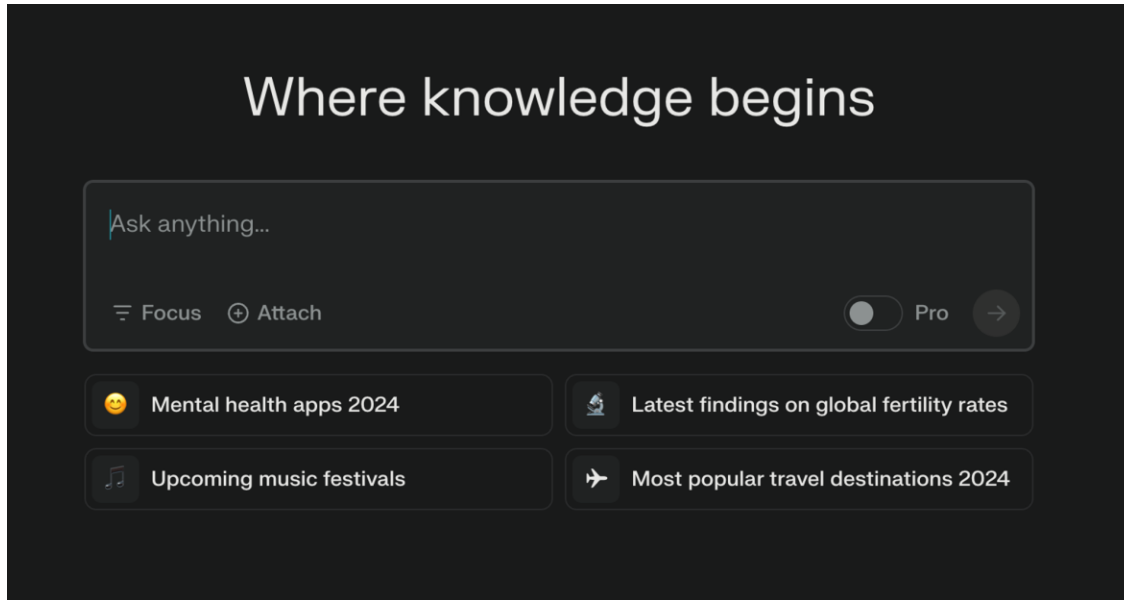
Prompting: Reject Work

Reject work.

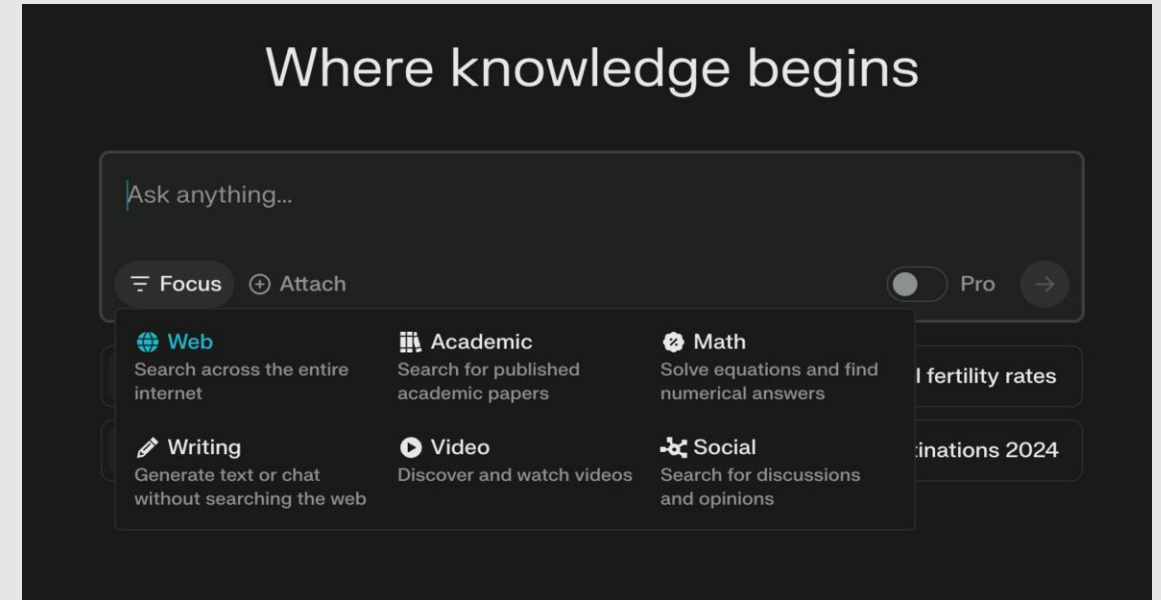
- Just like how we had to perform segmentations, I believe it's crucial to consider ways to enable the language model to reject work
 - **Explore if few shotting works well "Here are examples of similar questions we cannot answer..."**
- Give it permission to say no, but follow up, and set expectations
- If we model the success of this rejection system its just another precision recall trade off

Prompting: Showcase capabilities

Perplexity is always showing off capabilities trying to guide the user to behavior we perform well in



Example queries



Literally a list of capabilities, which likely have different prompts

Provide example and related queries

Focus on capabilities available and dynamically change prompting based on specific use case

Share and Copy buttons for feedback

what are ways to systematically improve rag applications

❖ Sources

Systematically improving RAG applications - YouTube
youtube - 1

Systematically Improving RAG applications - YouTube
youtube - 2

4 Steps to Production LLM: Climate RAG with Weave - YouTube
youtube - 3

View 5 more

Perplexity

Based on the video transcripts, here are some key ways to systematically improve RAG (Retrieval Augmented Generation) applications:

1. Implement feedback mechanisms and analyze user data:
 - Collect specific feedback and ratings from users on RAG responses
 - Adjust feedback prompts to enhance quality and quantity of user responses
 - Leverage minimal feedback through relevancy scores and embedding metrics
 - Use unsupervised learning techniques like BERTopic or LDA to extract meaningful topics from RAG queries
2. Analyze performance metrics:
 - Evaluate user satisfaction through discovered topic clusters
 - Correlate relevancy scores with user satisfaction to identify areas for improvement
 - Differentiate between topic queries and capability queries to target enhancements

Related video thumbnails: Evaluating the Search System, Systematically Improving RAG, TEST RAG APPLICATION, Back To Basics For RAG, IS JARGON CONFUSING YOUR SEARCH SYSTEM?, Evaluating RAG applications.

Special UI elements for sources by type (e.g., video), related queries, etc.

• Implement online monitoring for governance aspects like cost and latency

• Surface debugging errors and analyze qualitative traces

8. Conduct systematic benchmarking:

• Use evaluation datasets to benchmark performance

• Periodically optimize models and evaluations

9. Utilize evaluation frameworks:

• Use frameworks like RAGAS to evaluate both retrieval and generation aspects of RAG systems

• Assess metrics like faithfulness and answer correctness

By implementing these strategies, you can create a structured approach to continuously improve and refine your RAG applications based on real-world usage, user feedback, and systematic evaluation.

Share Rewrite

Related

- How can I leverage unsupervised learning to enhance RAG applications
- What are the best strategies for capturing implicit user feedback in RAG systems
- How can I utilize synthetic data to test RAG system improvements
- What role does accurate feedback play in improving RAG performance
- How can I differentiate between topic and capability queries in RAG applications

Related video thumbnails: Evaluating the Search System, Systematically Improving RAG, TEST RAG APPLICATION, Back To Basics For RAG, IS JARGON CONFUSING YOUR SEARCH SYSTEM?, Evaluating RAG applications.

Related queries

Monologues and Chain of Thought

I still see many companies underestimate the power of Chain of Thought

- Chain of Thought is often a 10% bump in performance
- It can often make the difference between something that is usable vs. unusable
- If we wrapped Chain of Thought in either XML or in streaming, we can:
 - Build a dynamic UI that renders the Chain of Thought as separate data
 - Treat the Chain of Thought as some kind of loading interstitial too!

Monologues and Chain of Thought

Overview:

- Leverage the monologue for multiple purposes
- When dealing with lengthy contexts, the LLM may struggle with recall or fully process instructions
- Try to prompt the model to reiterate relevant instructions and key text chunks before response generation
 - This is like training an intern who you'd naturally ask them to review and summarize important info
 - Consider including a “re-reading” prompt to improve reasoning

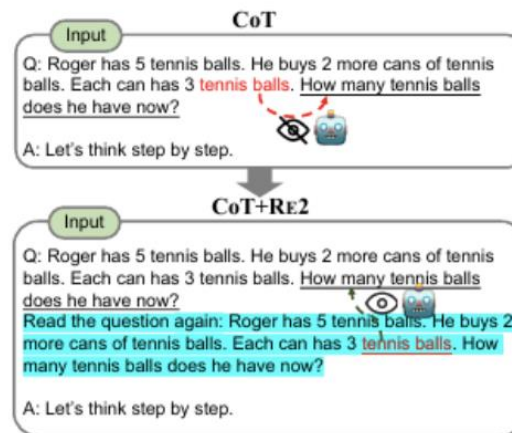
Re-Reading Improves Reasoning in Large Language Models

Xiaohan Xu^{1*}, Chongyang Tao², Tao Shen³, Can Xu²,
Hongbo Xu¹, Guodong Long³, Jian-guang Lou²

¹Institute of Information Engineering, CAS, {xuxiaohan, hbxu}@iie.ac.cn

²Microsoft Corporation, {chotao, caxu, jlou}@microsoft.com

³AAIL, School of CS, FEIT, UTS, {tao.shen, guodong.long}@uts.edu.au



```
from pydantic import BaseModel

class Response(BaseModel):
    relevant_instructions: str
    relevant_context: str
    chain_of_thought: str
    content: str
```


Case study: Generating Quotes for SaaS Pricing

Overview: We built a capability focused on generating pricing quotes based on a call transcript

- Detect that a pricing quote was being requested for a call
 - LLM Classifier was measured w/ recall
- In the prompt, we included the entire one-hour transcript and the entire pricing document
- By using monologues and Chain of Thought, we asked the LLM to:
 - Reiterate what variables determine the quotes
 - Reiterate relevant parts of the transcript
 - Reiterate parts of the pricing document that were relevant
- As a result, the LLM reasoned what pricing options might be available before generating a response. With a single prompt, we were able to get our pricing questions answered without complex multi-stage reasoning
- This allow us to make sure our follow ups were A+ and we had citations for our sales reps to verify the generated quotes. (We paid them to correct quotes, more data!)

Monologues before responses dramatically improve tonality and quality , which can be fine-tuned later without monologues.

Monologues and Chain of Thought (2/2)

Try to bake as much domain knowledge into these prompts, change prompts based on document types, be specific

You are an AI assistant tasked with answering queries based on given context.

Before generating a response, you must use `<monologue></monologue>` tags to reiterate the relevant instructions and the relevant text chunks involved in answering the query.

Here is the context you will be working with:

```
<context>
{{CONTEXT}}
</context>
```

When answering a query, follow these steps:

1. Use `<monologue></monologue>` tags to:
 - a. Reiterate the relevant instructions under `<relevant_instructions>` tags
 - b. Include the relevant parts of the context under `<relevant_context>` tags
2. After the monologue, generate your response and enclose it in `<response></response>` tags.

Here is the query you need to answer:

```
<query>
{{QUERY}}
</query>
```

Your output should follow this format:

```
<monologue>
<relevant_instructions>
[Reiterate the relevant instructions here]
</relevant_instructions>
<relevant_context>
[Include the relevant parts of the context here]
</relevant_context>
</monologue>
<response>
[Your answer to the query goes here]
</response>
```

Remember to always use the monologue tags before generating your response and ensure that your answer is based on the information provided in the context.

A single step is often not enough.

I would consider a validation pattern before going into full-blown multi stage agents.

As our LLMS get more powerful, we're going to be able to do more and more within a single prompt. What might take an agent now might be possible with a single prompt in the future.

Incorporating validators

- In latency-insensitive applications, incorporating validators can help **increase user trust and satisfaction** in your product
- **Just use evals / tests within the production workflow**
- When we have components like reasoning, citations, and text chunks, we can utilize them in a secondary prompt:
 - Use an external system to evaluate whether the reasoning, citations, and generated response **effectively answer the question**
 - If they don't, the system provides detailed feedback on what's incorrect, unreasonable, or needs revision
- Note!
 - These tests could be language models but it could also be unit tests or calls to external APIs

Incorporating validators: Referencing Content without Hallucinations

Problem:

- We wanted a language model to respond to emails with references to case studies and marketing material.
- However, we wanted to ensure that every single link is from the company namespace and that there were no hallucinations or invalid links since our links included UUIDs in the URL.

```
from pydantic import BaseModel, field_validator
import re
from typing import List

class Email(BaseModel):
    subject: str
    body: str

    @field_validator('body')
    def validate_urls(cls, v, context):
        allowed_domains = context.get('allow_list', [])
        urls = re.findall(r'http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[*\\(\)\,]|(?:%[0-9a-fA-F][0-9a-fA-F]))+', v)
        for url in urls:
            if not any(allowed_domain in url for allowed_domain in allowed_domains):
                raise ValueError(f"URL {url} is not in the allow list")
        return v
```

Incorporating validators: Referencing Content without Hallucinations

Solution:

- Our validator used a regular expression to find all urls
- Checked domains for our allow list
- We made a GET request to each URL to verify 200 status
- If there were any issues, we would send an error and request regeneration
- We initially had 4% failure rate, after 1 retry, it was 0%, after finetuning gpt-4o, 0% in a single pass

```
from pydantic import BaseModel, field_validator
import re
from typing import List

class Email(BaseModel):
    subject: str
    body: str

    @field_validator('body')
    def validate_urls(cls, v, context):
        allowed_domains = context.get('allow_list', [])
        urls = re.findall(r'http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[*\\(\)\,]|(?:%[0-9a-fA-F][0-9a-fA-F]))+', v)
        for url in urls:
            if not any(allowed_domain in url for allowed_domain in allowed_domains):
                raise ValueError(f"URL {url} is not in the allow list")
        return v
```

Fine-tune GPT-4o for free by Sep 23

OpenAI announces free fine-tuning for GPT-4o mini model

News

Jul 24, 2024 • 2 mins

Artificial Intelligence

Cloud Computing

Generative AI

Free fine-tuning allows OpenAI customers to train the GPT-4o mini model on additional data at no charge until September 23, starting with Tier 4 and Tier 5 users.

```
from openai import OpenAI
client = OpenAI()

client.fine_tuning.jobs.create(
    training_file="file-abc123",
    model="gpt-4o-mini",
    hyperparameters={
        "n_epochs": 2
    }
)
```

The screenshot shows the OpenAI platform interface for creating a fine-tuned model. The 'Base Model' is set to 'gpt-4o-mini-2024-07-18'. Under 'Training data', the 'Upload new' option is selected, with a file upload area labeled 'Upload a file or drag and drop here (jsonl)'. Under 'Validation data', the 'None' option is selected. The 'Suffix' is set to 'my-experiment'. The 'Seed' is set to 'Random'. Under 'Configure hyperparameters', 'Batch size', 'Learning rate multiplier', and 'Number of epochs' are all set to 'auto'. The interface includes a sidebar with navigation options like 'Dashboard', 'Assistants', 'Fine-tuning', 'Batches', 'Storage', 'Usage', and 'API keys'. The top navigation bar includes 'Playground', 'Dashboard', 'Docs', 'API reference', and a user profile icon.

@jxnico

maven.com/applied-llms/rag-playbook

Homework



Finish homework from last few weeks

- Generate synthetic data to test your system
- Analyze user queries through topic modeling and use this to inform inventory or capabilities issues
- Implement user feedback mechanisms
- Focus on building more robust retrieval indices based on entity types (e.g., documents, images, text-to-SQL)
- Focus on improving query routing systems (e.g., which search methods do you want to work better)
- Improve representations for each sub-task. Consider preparing triplet data sets, using Cohere re-rankers, or finetuning an embedding model (with sufficient data)



Questions to ask yourself

- Am I being too subtle with collecting feedback on my product.
- Could building better citations help me gain user trust and satisfaction?
 - Is there any way for me to leverage the citations to collect more relevancy data?
- Could I implement better streaming, interstitials, and follow-up actions to make my application feel faster?
- How can I better promote capabilities and reject other work
- Can I include monologues and chains of thought to reiterate parts of the prompt and improve reasoning in my system.

Agenda

Collecting feedback

Building streaming

Prompting

Course conclusion

Q&A

Course conclusion

What I hope to distill into you

- Evaluations are what you need to understand how to improve.
 - Evaluations are a dataset that can inform your decisions.
 - They power few-shot examples, which can be used with retrieval to improve performance.
 - They are also the finetuning datasets... (Train vs. Test Split)
- Synthetic data with large language models and customer feedback from engagement are two sides of the same coin
 - It's all data and data augmentation.
- These are the fundamental building blocks of creating successful machine learning products.
 - If you refuse to believe this, you're condemning yourself to being lost and confused in this hyped landscape.

The process:

- A good product generates better evaluations.
- Better evaluations allow you to train / finetune models to create an even better product.
- Data analysis over segments of your evaluations tells you where to focus your product development efforts.

This marks the end of our course.

Please don't hesitate to give any feedback.

My goal is to convey to folks the importance of having strong fundamentals. If you think there's any way this course can be made better for future iterations, please let me know.

Agenda

Collecting feedback

Building streaming

Prompting

Course conclusion

Q&A

Offices Hours tomorrow:

#general for dory link
#ask-the-instructors